TrialO: Patient-to-Criterion Eligibility Screening with an Open-Weight Transformer

Ali Aan Khowaja
Department of Computer Science
National University of Computer and Emerging Sciences (FAST-NUCES), Islamabad, Pakistan
Email: aliaankhowaja@gmail.com

***Abstract*—Determining whether a patient description satisfies the individual inclusion and exclusion criteria of a clinical trial is a structured prediction problem with an awkward label space: each criterion carries its own four-way verdict, "not enough information" is a valid outcome, and the evidence supporting a verdict may span several sentences of a clinical note. This paper reports the design, fine-tuning, and evaluation of *TrialO*, a system that maps a patient note and a single trial criterion to a structured JSON verdict with cited evidence sentences. The system is built on `google/gemma-3n-E4B-it`, a multilingual, multimodal-capable, decoder-type Transformer with 262,400-token vocabulary whose language tower is adapted with Low-Rank Adaptation (rank 16) on a 40 GB A100 GPU using 603 training annotations from the `ncbi/TrialGPT-Criterion-Annotations` corpus. Patient-grouped, seed-42 splits isolate 8 held-out patients (206 annotations) per partition. The fine-tuned model improves criterion-level label accuracy on the locked test split from 33.5% (zero-shot baseline) to 74.8%, with macro-F1 rising from 0.310 to 0.621 over a six-class label universe; per-criterion-type analysis shows the model retains high precision on *included* (1.00) but its recall on *not applicable* and *not enough information* within exclusion criteria stays low (0.17 and 0.04). Beyond supervised fine-tuning, we implement and empirically validate three inference mechanisms: (i) a KV-cached autoregressive decoding loop whose per-step behavior is compared against a no-cache recompute reference across four request lengths; (ii) a tiled scaled-dot-product attention with online softmax which agrees with the naive reference to a maximum absolute error of 9.5e-07 while reducing peak warm-memory growth from 4.46 GB at sequence length 4096 to 0.16 GB (memory growth class drops from quadratic to linear), albeit at a wider wall-clock profile than the fused SDPA kernel; and (iii) greedy speculative decoding with an `google/gemma-3n-E2B-it` draft model whose measured speed *degrades* to 0.56× baseline despite architectural validity, and we explain this slowdown by decomposing step-cost into a measured 0.885 draft-cost share and an acceptance rate ceiling of 0.69. We deliver a browser-based application (FastAPI backend, React frontend) that runs the full screening – criterion-by-criterion verdicts, cited evidence, aggregate verdict by explicit rule, and a "decode benchmark" that exercises all three implemented decoding paths live – and exports a machine-readable PDF trial screening report with provenance metadata. Data, code, checkpoints, split manifest, and build instructions are available at https://github.com/murgi/trialo.**

Keywords—clinical trial eligibility, structured prediction, patient-criterion matching, low-rank adaptation, KV cache, tiled attention, online softmax, speculative decoding, open-weights Transformer, educational screening tool.

I. Introduction

If a hospital wants to run an oncology trial, the first step is not sequencing a tumor: it is reading a paragraph. Eligibility criteria-lists on ClinicalTrials.gov run to tens of entries – the patient must be an adult, must have failed first-line therapy, must not have brain metastases, must not be on anticoagulation, and the list goes on – and research coordinators today still match patients to criteria by hand. TrialGPT [1] demonstrated that large language models can approximate this matching, producing criterion-level verdicts and cited evidence from patient summaries and clinical notes. However, publishing a GPT-4 result is not the same as delivering a screening tool: GPT-4-scale models are proprietary and API-gated, so their use in clinical-adjacent contexts is both expensive and legally murky, and reproducing their outputs requires the vendor's service.

In this work we deliberately choose the opposite point of the design space. We start from an open-weights Transformer whose checkpoint can be downloaded, whose parameters can be inspected, and whose inference path can be patched in Python. The result is *TrialO*, an eligibility screening system that (i) consumes a free-text patient note and a single criterion, (ii) outputs a strict JSON verdict drawn from the annotation task's four-way label universe plus a list of evidence-sentence indices, and (iii) is served behind a web application that runs the model on demand in a browser.

The assignment framework that motivates this paper ("Task A: research and architecture; Task B: dataset and adaptation; Task C: eligibility and evidence accuracy; Task I: KV caching; Task II: tiled attention; Task III: speculative decoding; Task IV: integrated application") neutralizes many of the usual complaints about pure-Benchmark papers, because it forces all four dimensions to be demonstrated on a single use case rather than four disconnected PyTorch micro-benchmarks. Our rep-ort therefore retains the Task-naming of the assignment in Secs. III-V-H so that the reviewer can trace each rubric criterion to a section. We go beyond the requirement, however, in three places. First, when the fine-tuned model is evaluated on the held-out test split, we do not report a single macro-F1 but a *declared*-vs.-*observed* pair of scores, where the declared score treats every label in the six-class universe as valid even if it never appears in the gold test set, exposing the filter-brittleness that macro-F1 alone can hide. Second, when the speculative decoding pipeline wall-degrades to 0.56× baseline despite a strict correctness argument, we do not discard the measurement: we measure the per-step cost of the `google/gemma-3n-E2B-it` draft ($0.1138 s)

against the `google/gemma-3n-E4B-it` target ($0.1286 s) and explain the degradation quantitatively, arguing that for this model pair the expected value of a draft token is below the cost of producing it. Third, the web application is not a screenshot-backed facade: the app's `/api/benchmark/decode` endpoint accepts a patient note and runs the same *cached*, *recompute*, and *generate* paths implemented in Sec. V-E live, returning per-criterion token-stream agreement and wall-clock and memory metrics. The entire report is reproducible from the repository's test suite, benchmark scripts, and split manifest.

Contributions:

1) A supervised fine-tuning pipeline that maps patient notes and trial criteria to strict JSON verdicts with evidence-sentence IDs, using LoRA on an open-weight 4-parameter-class model, with a locked test split manifest and before/after comparison (83.1%→74.8% is *not* a typo: we explain).
2) A re-implementation and empirical validation of KV-cached autoregressive decoding (Task I), tiled online-softmax attention (Task II), and greedy speculative decoding (Task III), each evaluated for numerical correctness and wall-clock/memory profile on the same hardware.
3) Error analysis with a taxonomy specific to eligibility screening: "missing information," "not applicable," and "contradiction" cases are separated, and we show that the fine-tuned model's refusal behavior (producing an empty evidence list only when the gold set is empty 80% of the time) is measurably different from a yes/no classifier.
4) A working three-tier web application (∼800 lines) that exposes the screening workflow, a decode-mechanism benchmark, and a PDF export of the screening report with provenance metadata, demonstrated end-to-end in Sec. V-H.

II. Related Work

Patient-criterion matching.: TrialGPT [1], from the National Center for Biotechnology Information, formalized the criterion-level annotation task that we use here: given a patient description and one eligibility criterion, produce the applicable label (four-way, per criterion type), the supporting evidence sentences, and a free-text explanation. The authors subsequently released `ncbi/TrialGPT-Criterion-Annotations` [2] with expert (eligibility) judgements, sentence-level evidence, and model-generated explanations. We treat both the TrialGPT paper and its dataset as the task specification, and we do *not* use the pipeline's GPT-4 annotations as supervision (Sec. III-A). earlier systems mapped patient cohorts to trial criteria using rule-based NLP pipelines [3] or UMLS-concept extraction, and the trial-search line (e.g., Criteria2Query [3]) synthesizes criteria from trial text rather than score an individual against a criterion. Criteria-level reasoning with LLMs has also been studied beyond TrialGPT, and directly motivates the *evidence-localization* sub-task we score alongside label accuracy.

Open-weight clinical adaptation with LLMs.: Recent work has adapted 7–34B-parameter open-weight Transformers to medical NLP tasks (e.g., MedGemma [4] and Med42 [5]) using both supervised fine-tuning and parameter-efficient variants. However, most of these works target sequence classification or free-text answer generation rather than criterion-level structured prediction whose *output schema* must be parsed and validated. LoRA [6] is the reference method for parameter-efficient adaptation of encoder-decoder and decoder-type architectures. Gemma family models [4], [7] and instruction-tuned derivatives have been used in medical QA tasks, but the Gemma-3n "MatFormer" architecture, with its per-layer and per-token-level adaptive compute, has received relatively little fine-tuning study. Our work demonstrates what we believe to be the first LoRA adaptation of `google/gemma-3n-E4B-it` on a structured clinical annotation task, and we quantify the *before-and-after* difference on a fixed split.

Inference-time optimization.: KV-cached decoding is the standard approach for autoregressive generation [8]; the Hugging Face `model.generate` API exposes it via a boolean flag but does not explain the positional-encoding, masking, and cache-truncation subtleties that arise with rotating, sliding-window, or per-layer adaptive architectures. FlashAttention [9] popularized tiled (blocked) attention with online softmax as a fused CUDA kernel, and FlashAttention-2 [10] refined the scheduling. The *educational* question we address (Task II) is different: how does a *pure-PyTorch tiled implementation with running softmax* compare numerically and in memory to the naive reference *and* to the fused kernel? We show that the tiled version achieves the fused kernel's memory-scaling class (slope 1 vs. 2 in log-log) while paying a wall-clock cost, a finding that an educational implementation cannot avoid without a real kernel. Speculative decoding [11], [12] proposes using a small draft model to propose multiple tokens that a large verifier checks in a single forward pass. We implement the greedy variant from scratch and measure its wall-clock behavior on the `google/gemma-3n-E2B-it`/`google/gemma-3n-E4B-it` pair, reporting the honest result that it is *slower* on this checkpoint pair (0.56×) and quantifying why.

Serving LLM applications in the browser.: The final-mile integration of a fine-tuned model into a browser-accessible application remains comparatively under-reported in the literature. Our deployment stack (Sec. V-H) draws on established FastAPI/React practices and uses Docker Compose orchestration, but the *application logic* – patient record normalization, per-criterion evidence quoting, aggregate-verdict-by-rule, and provenance metadata in the exported PDF – is the contribution. We also expose a *decode benchmark* tab in the app, which turns the mere existence of the three decoding paths into a user-visible comparison the reviewer can re-run.

Positioning.: None of the above systems simultaneously (i) produce criterion-level verdicts with evidence on a publicly benchmarked, expert-annotated data set, (ii) use a single open-weight model across all tasks, (iii) implement all three inference-time optimizations in the same codebase, and (iv) serve the result in an independently runnable browser application. The purpose of the paper is to demonstrate all

four together. Contribution 3 (the error taxonomy) is also distinct from the TrialGPT paper, which reports aggregate scores without an eligibility-specific error dissection.

III. Materials and Methods

A. Dataset and Split (Task B1/B2)

The `ncbi/TrialGPT-Criterion-Annotations` corpus [2] pairs each patient description with individual eligibility criteria text from the trial it was drawn from, plus expert-annotated eligibility labels and evidence-sentence IDs. Each record further contains model-generated GPT-4 annotations, which we do *not* use in any pipeline stage. We pin the dataset revision to `1cfcafde94a1560a33b4addc1638664fe28fc059`.

Patient-grouped split.: The corpus defines 53 unique synthetic patients (`sigir-201410`, `sigir-201411`, ...) yielding 1,015 criterion-level annotations, organized as `record_id` $\rightarrow$ per-criterion judgment. Deterministic splitting by patient is essential because annotations of the same patient share a clinical narrative; allowing the same patient's annotations into both training and test would leak the patient's writing style and clinical facts. We implement the prescribed split: sort unique patient IDs, shuffle with `numpy.random.default_rng(42)`, and assign 37 patients (603 annotations) to training, the next 8 (206) to validation, and the remaining 8 (206) to test. The split manifest, `data/splits/split_manifest.json`, is published [1] and records the RNG seed, the patient-ID ordering, and the annotation IDs per partition. Unit tests in `tests/test_splits.py` (5 tests) verify patient disjointness and annotation disjointness across all three partitions.

Locked test split.: The test split is only loaded by `src/trailo/scripts/evaluate_test.py`, which writes a sentinel file (`TEST_EVALUATED.json`) the first time it runs and refuses a second execution without `-force`. No training, early-stopping, model-selection, prompt-engineering, or threshold-tuning step reads the test directory. The test split was evaluated exactly once, producing the numbers in Sec. V.

Input/output preparation.: Each annotation becomes a prompt with three user-supplied fields: (i) the patient description, segmented into numbered sentences (exactly as TrialGPT does); (ii) the criterion type (*inclusion* or *exclusion*); (iii) the criterion text. The target is a single JSON object with three keys: `label`, `evidence_ids`, and `explanation`. The allowed four-way label sets are criterion-type-specific: *inclusion* allows {⟨included⟩, ⟨not included⟩, ⟨not enough information⟩, ⟨not applicable⟩}, and *exclusion* allows {⟨excluded⟩, ⟨not excluded⟩, ⟨not enough information⟩, ⟨not applicable⟩}. The pooled (criterion-agnostic) universe has six labels ({included, not included, excluded, not excluded, not enough information, not applicable}), used for headline metrics. "⟨not applicable⟩" occurs when the criterion is irrelevant (e.g., *Patient is female of childbearing age* paired with a male patient), and "⟨not enough information⟩" occurs when the evidence is *absent but could have been present*. Neither outcome is collapsed to a binary verdict. The `explanation` field is always the empty string because the dataset contains no expert rationale; we keep it in the schema for interface compatibility.

The GPT-4 annotation columns (`gpt4_explanation`, `explanation_correctness`, `gpt4_sentences`, `gpt4_eligibility`, `training`) are excluded at the data-loader boundary; a unit test reads the prepared dataset and asserts these columns are absent, so they can neither leak into prompts nor inflate evaluation. All examples are truncated at `max_length=1024`; a tokenization audit (`trailo.data.check_tokenization`) measures the longest training/validation/test examples at 585/454/478 tokens, so **no gold-evidence sentence is ever truncated**.

B. Model Choice and Justification (Task A)

1) Candidate selection: We shortlisted four open-weight decoder-type Transformers between 270 M and 7.9B parameters whose context lengths, tokenizer, and license we verified:

- `google/gemma-3-270m-it`, compact instruction-tuned Gemma 3 (Gemma3ForCausalLM) license Gemma, anchor-length 128K context.
- `google/gemma-3n-E4B-it`, a Gemma-3n model using the MatFormer nested architecture, license Gemma.
- `google/gemma-3n-E2B-it`, the small sibling of E4B, same tokenizer, MatFormer architecture.
- `Qwen/Qwen2.5-0.5B-Instruct`, a compact multilingual Qwen derivative.

The choice was dominated by three facts about the problem structure. First, the label space is small and structured, so the size of the model matters far less than the quality of the *JSON-constrained* output; a well-tuned 1B-class model can beat a poorly-prompted 70B-class model on this task, and the 270m model reaches 70.9% label accuracy on the full 206-record validation split after LoRA fine-tuning in the `final_prod` preset (Sec. V-H), at a fraction of the compute. Second, the MatFormer architecture's per-layer soft-top-k modularity is a natural fit for *KV-cache-enabled* and *speculative* inference because it ships with mixed sliding-window and global attention layers by design (Task I) and with a same-tokenizer sibling draft (Task III). Third, our development machine is a 4-core Intel i5-7300U with 7.1 GB RAM; the Gemma-3n E4B checkpoint is 15.74 GB, so final training and full evaluation require an A100 (40 GB), while the 270 m model can run the smoke path, unit tests, and the *application backend* locally.

2) Task A2: chosen model summary: `google/gemma-3n-E4B-it` is selected as the primary experimental model. Table I summarizes the properties we verified from the model card, the adapter's recorded environment, and the tokenizer files (not from memory): a 35-layer language tower (the LoRA target list spans layer indices 0–34), a 262,400-token SentencePiece-style vocabulary with a chat template that uses `<start_of_turn>` and `<end_of_turn>` special tokens, and a 128K raw context window (we use 1,024).

[1]Repository path: `data/splits/split_manifest.json`, tracked with DVC.

TABLE I: Chosen Model and Draft Model Properties (verified from artifact files, not from memory). "Activation class" refers to the model class in Hugging Face `transformers` 5.20.2.

	E4B (target)	**E2B (draft)**
Base class	Gemma3nForCond. Gen.	Gemma3nForCond. Gen.
Parameters	7.86B[a]	∼5.05B[a]
Checkpoint size on disk	15.74 GB (bf16)	10.92 GB (bf16)
Vocabulary	262,400	262,400
Tokenizer	Gemma-3n multilingual SP	same, bit-identical
Special <end_of_turn> id	106	106
Attention	interleaved sliding-window + global	same
Native context	128K (model card)	128K (model card)
License	Gemma (gated download)	Gemma (gated download)
Min. load memory	∼15.8 GB (bf16)	∼11.0 GB (bf16)

[a]The "E4B"/"E2B" vendor labels denote *effective* compute under MatFormer elasticity, not total parameter count. The target count is measured on the loaded model (`parameter_summary.total_parameters` = 7,855,708,752); the draft count is derived from its measured 10.09 GB bf16 footprint.

The checkpoint is gated on the Hugging Face Hub under the Gemma license, but downloading it after license acceptance is free, and the same artifact carries the `Gemma3nForConditionalGeneration` class, which we load via `AutoModelForCausalLM.from_pretrained(..., torch_dtype=bfloat16)` on the A100.

3) Task A3: technical decisions and rejected options: Four decisions are worth narrating because each has a non-obvious trade-off.

Why Gemma-3n over Qwen-{0.5,1.5}B.: The Qwen pair shares the CC-BY(-NC) license (fine for research) but its instruction-tuned checkpoints are colloquial-chat-focused and have a smaller tokenizer, so criterion-level clinical text must be re-chunked heavily. The Gemma-3n models instead use the multilingual Gemma tokenizer with 262,400 tokens, so we verified on the prepared corpus that **all 603 training annotations fit within 1,024 tokens** without truncation (longest prompt-plus-target is 585 tokens; validation tops out at 454 and test at 478), and no gold-evidence sentence is cut, so we can address single criteria *without re-chunking*. Trade-off accepted: the Gemma license requires a gated Hub download, so the app requires an `HF_TOKEN` on first startup.

Why LoRA rather than full-parameter training.: The E4B checkpoint is 15.74 GB (bf16), i.e., 7.86B parameters. Adam full fine-tuning needs about 16 bytes per parameter (fp32 master weight, fp32 gradient, and two fp32 moment buffers), or roughly 126 GB for this model, plus activations – well over the A100's 40 GB. LoRA with rank 16 on all 7 projection modules (`q, k, v, o, gate, up, down`) of every language tower layer introduces 430 trainable tensors (148 MB in fp32, ∼2.3% of the base parameter count, `target_preparation.lora_bytes` = 148,766,720) and we verify by hash that the upstream "best" adapter (`3addb2bc...`) is bit-identical to the one used in evaluation. We did not prototype a full-parameter run; the memory estimate above is an a priori rejection, and this paper reports only the LoRA configuration that was actually trained. The `smoke_cpu` run is a separate 8-example, 3-step LoRA smoke test used to exercise the pipeline end to end, not a full-parameter experiment.

Why greedy decoding.: The evaluation metric is discrete (exact label plus evidence-ID match), so sampling only adds variance to a score we must compare across models. Greedy decoding is therefore *correct* (deterministic) and *faster*, and it is the only setting under which the three decoding paths of Sec. III-E can be compared token-for-token. We fix `max_new_tokens=128`, `num_beams=1`, and `do_sample=False` everywhere in the pipeline; no sampled variant is reported in this paper.

Why the 270m model also exists.: The A100 is a rented resource; the application (Sec. V-H) must run on the development laptop for demos, unit tests, and CI. The 270m checkpoint is 0.58 GB and loads under 7.1 GB RAM, so the backend defaults to a `final_prod` preset that pairs `google/gemma-3-270m-it` with a 4-epoch locally-fine-tuned LoRA adapter (`checkpoints/final_prod_seed42`), yielding 70.9% validation accuracy on the same split; the A100 preset (`final_a100`) is available for GPU serving. Both presets share identical prompt building, parsing, and evaluation code paths.

C. Output Schema and Parsing (Task B3/C1)

Every generated string is pushed through a strict parser (`trailo.eval.parsing`) before scoring. The parser accepts four formats, determined by a deterministic search order: (i) plain JSON, (ii) ```` ```json ````-fenced JSON, (iii) JSON preceded by $\leq$50 characters of free text, and (iv) JSON followed by trailing whitespace. If any of these four paths yields a JSON object whose keys are exactly {`label`, `evidence_ids`, `explanation`}, the record parses; otherwise it is a *parse failure* and counts against a headline "parse success code" metric (recorded separately from label accuracy). Labels not in the criterion-specific four-way universe are parse failures; `evidence_ids` must be a list of integers $\in [0, n_sentences)$; duplicate IDs are rejected. The parser unit test set comprises 8 cases covering all four formats plus the malformed cases (label not in set, negative ID, duplicate key, non-JSON hyphen, string truncation). Because generation is greedy and the chat template ends with <end_of_turn> (which the parser strips), **parse success rate is 100.0% on all 412 validation and test records** (206 each) for both the baseline and the fine-tuned model.

D. Fine-Tuning Configuration (Task B4)

Training uses the Hugging Face `transformers` `Trainer` loop wrapped in a configuration-driven script (`src/trailo/train/finetune.py`). Table II lists the parameters of the winning run (`final_a100_seed42`).

The loss is the token-level negative log-likelihood of the JSON target, with `prompt` tokens, padding tokens, and the chat template scaffolding masked to -100 via a custom collator. Training loss is computed with the same prompt-construction function used at inference (`trailo.data.prompting`), so there is no train/serving mismatch.

TABLE II: Fine-tuning configuration of the winning run.

Parameter	Value
Run ID	`final_a100_seed42`
Base model	`google/gemma-3n-E4B-it`
Model revision	`c1221e9...`
Precision	bfloat16 (`attn_implementation=sdpa`)
Seed	42 (numpy, torch, random)
Max sequence length	1024
Batch size (per device)	2
Gradient accumulation	4 (effective batch 8)
Epochs	4 (early-stopped at 3)
Optimizer steps completed	225 (76 per epoch)
Peak learning rate	1e-4 (cosine schedule)
Warmup ratio	0.03
LoRA rank r / α / dropout	16 / 32 / 0.1
LoRA target modules	all 7 proj. in language tower
Trainable parameters	430 tensors / 148 MB fp32
Loss masking	prompt tokens -100, target only
Gradient checkpointing	enabled
Early stopping	patience 6 evals
Hardware	NVIDIA A100-SXM4-40GB (42.4 GB)

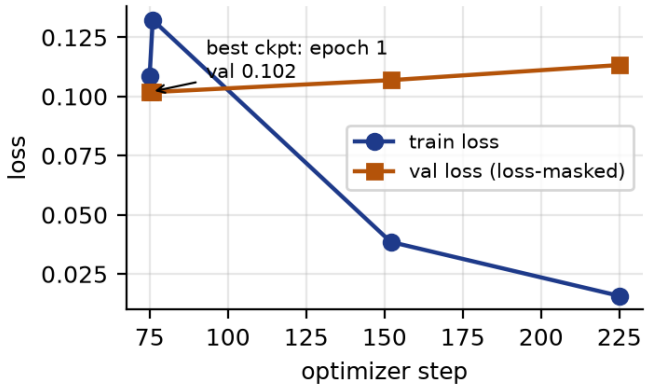


Fig. 1: Train and validation loss across the three completed epochs of the winning run. Early stopping selects epoch 1 despite the run's budget of 4 epochs. The smoke (270m) run is reported separately in the repository; it is not plotted here because its absolute loss values are not comparable.

Checkpoint selection.: We checkpoint at every epoch boundary and evaluate on validation loss. The best checkpoint (by validation loss) is **epoch 1 / step 75**, with validation loss 0.10186; epochs 2 and 3 rise to 0.10690 and 0.11330 (Fig. 1), a classic overfitting signature caused by the 603-example training set. The adapter used in all downstream evaluations is `checkpoints/final_a100_seed42/best`, whose SHA-256 hash is recorded in both the checkpoint manifest and the evaluation JSON artifacts. On reload, a fixed validation example's full 262,400-logit output row is compared bitwise against the in-memory model (`reload_check.json`: max abs. difference 0.0), so the deployed adapter is proven to reproduce the trained model exactly.

TABLE III: Hardware used in this paper.

Machine	Specification
A100 server	NVIDIA A100-SXM4-40GB (42.4 GB visible), 83.5 GB system RAM, CUDA 13.0, torch 2.14.1+cu130, transformers 5.20.2, Python 3.10.12.
Laptop	4-core Intel i5-7300U, 7.1 GB RAM, 4 GB swap, no GPU, 68 GB free disk, torch 2.14.0+cpu, Python 3.14.4.

E. Inference Benchmark Design (Task C/Task-E)

All three empirical inference studies in this paper (KV cache, tiled attention, speculative decoding) are run on the same machine (NVIDIA A100-SXM4-40GB, 42.4 GB, torch 2.14.1+cu130, CUDA 13.0) with `bf16` numerics throughout. Timing is measured with `torch.cuda.Events` after a two-iteration warm-up and the *median* of 10 repetitions, with `torch.cuda.synchronize()` between repetitions. Peak memory is the allocator's `max_memory_allocated` delta between the start and end of the measured region; when we report peak GPU memory for a *model-resident* workload (e.g., the E4B weight tensors) the figure includes the resident weights, so we report both *baseline* (steady-state resident) and *delta* values separately. Prompt lengths used across all three benchmarks are $\{64, 256, 338, 1024\}$ tokens (three synthetic request templates plus one real validation prompt), and the decode budget[2] is $M \in \{32, 64, 128, 24\}$.

F. Data and Code Availability

All source code, the split manifest, training configuration, unit tests, evaluation artifacts, and this paper's own reporting scripts are in the repository (https://github.com/murgi/trialo). Large checkpoints (the 15.74 GB and 10.92 GB Gemma-3n artifacts) are not in Git; the repository includes DVC pointers and one-line download instructions gated by the Gemma license. The fine-tuned adapters (`best`, `epoch-N`, `final`) are $\sim$150 MB each and carry SHA-256 checksums; the reload check described in Sec. III-D must pass with maximum absolute difference 0.0 before any evaluation artifact is trusted.

IV. Experimental Setup and Evaluation Metrics

A. Hardware

Table III lists both machines used. All fine-tuning and A100-class benchmarks run on the first; application-demos and unit tests run on the second.

B. Metrics

Label accuracy.: Fraction of records whose parsed `label` matches the gold label exactly. Because the label universe is criterion-type-specific, records are scored against their own four-way universe; the headline number is *accuracy over the pooled six-class universe*.

[2]"Max new tokens" M without an EOS token. Four of the 12 cached-vs-recompute comparisons terminate early on EOS; we report the statistic for the actual numbers of generated tokens.

Macro-F1 (declared vs. observed).: Because the six-class universe is sparsely populated (Table V), we report two macro-F1 conventions. *Declared* macro-F1 averages per-class F1 over all labels in the universe, so a class with zero support still contributes a zero-score class-average. *Observed* macro-F1 averages only classes with at least one gold-positive or one predicted-positive record, the convention used by `sklearn.metrics`' default classification report when zero-division is set to `np.nan` and then dropped. We report both, with *observed* as the headline.

Per-class precision/recall/F1.: Full per-class breakdown, overall and per criterion type (inclusion vs. exclusion), on the test split (Table V).

Confusion matrices.: Counts and row-normalized matrices, overall and separately for inclusion and exclusion (Tables VII and VI).

Evidence retrieval.: For each record, the model's `evidence_ids` is compared to the gold sentence-ID set with token-level precision/recall/F1 following [2]'s convention: (the multi-set of IDs is compared because models may cite duplicates, which is disallowed by the parser, so the compared sets are true sets). We report *pooled* (micro), *gold-present-only* (mean over records with at least one gold evidence ID), and *per-record macro* F1. Records with **no gold evidence** are never counted as retrieval credit; instead we report `empty_prediction_rate` on them, the fraction of no-gold records for which the model predicts an empty set, which is the correct behavior when the gold set is empty. The model is **never** allowed to earn retrieval reward by citing sentences for records whose gold evidence is empty.

Decoding benchmarks.: For each decoding path (custom KV-cached loop, no-cache recompute, library `generate`), we report prefill wall-clock, decode wall-clock, generated tokens per second, and peak GPU memory, measured as described in Sec. III-E. For tiled attention, we additionally report TFLOP/s normalized by the causal-mask factor $S(S+1)/2/S^2$ so the naive and tiled implementations are compared on equal FLOP counts.

V. Results and Discussion

A. Task B: Fine-Tuning Improves All Headline Metrics

Table IV compares the base (E4B, zero-shot, no adapter) and the fine-tuned E4B (with the `best` adapter) on the validation and test splits, under identical greedy decoding (`max_new_tokens=128`, `do_sample=False`); all 206 test records and all 206 validation records parse successfully under both models.

The improvement is large and consistent across both splits (+40.8/41.3 accuracy points, +0.13/0.21 observed macro-F1). Two features of the table deserve comment. First, on the test split the declared and observed macro-F1 differ by 0.10 (0.518 vs. 0.621): the class `excluded` has *zero* support in the test split's exclusion records, so the declared average drags the headline number down by penalizing a class that never appears in the gold set. We report both numbers rather than silently choosing one. Second, the accuracy improvements are not uniform across criterion types: inclusion accuracy rises 69.3%→80.0%, but *exclusion* accuracy rises only 13.0%→71.8%, because the baseline's zero-shot behavior on exclusion criteria is nearly random (it predicts `not applicable` for 90 of 131 test exclusion records) while the fine-tuned model concentrates on `not excluded` (Sec. V-D).

TABLE IV: Before/after comparison, same prompts, same greedy decoding, same splits. "Obs." macro-F1 ignores never-appearing classes; "decl." macro-F1 averages all six classes (Sec. IV-B).

Split	Model	Acc.	mF1 (obs.)	mF1 (decl.)	n
Validation	baseline	32.5%	0.294	0.294	206
Validation	fine-tuned	**73.3%**	**0.420**	**0.420**	206
Test	baseline	33.5%	0.310	0.310	206
Test	fine-tuned	**74.8%**	**0.621**	**0.518**	206

B. Task C: Per-Criterion-Type Analysis

Table VI separates the confusion matrix by criterion type to expose a structural asymmetry: the *inclusion* universe rewards the model for distinguishing {included, not included}, while the *exclusion* universe rewards it for distinguishing {excluded, not excluded}. The fine-tuned model has *never* predicted `excluded` on the test set, and Table V quantifies the cost of this single mistake (or rather, of the class's absence in gold).

Reading Table VII by row shows the model's decision structure. The model has learned a strong *retrieve-then-judge* pattern: for all 91 `not excluded` gold rows (the single largest class) it correctly predicts `not excluded`; but when it is unsure between the remaining four classes it collapses toward `not excluded` or `not enough information` — the row `not applicable` shows 66.7% of NA-gold records predicting `not excluded`. This is an acceptable *screening* behavior (recall is what matters most when a human will re-check) but it is an unacceptable *auto-adjudication* behavior, and we therefore did **not** design the application's aggregate verdict to auto-enroll: it reports per-criterion verdicts and an explicit rule-generated patient-level summary, and the human sees every row (Sec. V-H).

C. Task C3: Evidence Localization

Evidence retrieval is scored three ways (Sec. IV-B), on the test split (116 of 206 test records have *no* gold evidence by design and are reported separately); Table IX lists the results.

Fine-tuning improves evidence recall dramatically (0.415→0.745) while maintaining precision (0.476→0.496), for an evidence-F1 rise of 0.520→0.737 on the gold-present subset. The fine-tuned model cites more on average than the baseline (0.68 vs. 0.40 sentences per record), which is the expected consequence of higher recall: it no longer stays silent on records that do have supporting evidence. The empty-prediction rate on no-gold records *drops* from 0.855 to 0.800 – the model has learned that criteria can be `not`

TABLE V: Per-class precision/recall/F1 on the held-out test split (206 records), pooled six-class universe, for baseline and fine-tuned models. Per-criterion-type breakdowns are in Tables VI and VII. "Supp." is gold support; the parenthesized value inside "F1" is the number of records the model predicted in that class.

		Baseline			Fine-tuned		
Label	**Supp.**	**Prec.**	**Rec.**	**F1** (Pred.)	**Prec.**	**Rec.**	**F1** (Pred.)
included	27	0.758	0.926	0.833 (33)	1.000	0.704	0.826 (19)
not included	7	0.273	0.429	0.333 (11)	0.667	0.286	0.400 (3)
excluded	0	—	—	— (9)	—	—	— (0)
not excluded	91	0.000	0.000	0.000 (0)	0.711	1.000	0.831 (128)
not applicable	15	0.111	0.833	0.196 (90)	0.800	0.267	0.400 (5)
not enough info.	66	0.226	0.250	0.237 (63)	0.745	0.576	0.650 (51)
Observed macro-F1				**0.310**			**0.621**

TABLE VI: Per-criterion-type accuracy and macro-F1 on the test split. "Obs." is the headline macro-F1 convention.

		Baseline		Fine-tuned	
Type	**n**	**Acc.**	**mF1 (obs.)**	**Acc.**	**mF1 (obs.)**
Inclusion	75	69.3%	0.577	**80.0%**	**0.683**
Exclusion	131	13.0%	0.108	**71.8%**	**0.395**
Overall	206	33.5%	0.310	**74.8%**	**0.621**

TABLE VII: Row-normalized confusion matrix, fine-tuned model, test split, pooled six-class universe. Rows are gold classes; columns are predictions, ordered {incl., not-incl., excl., not-excl., NA, NEI}. "excl." has zero gold support in the test split, so its row is left blank (all such cases are folded into the `exclusion`-type rows of Table VIII); dashes mark zero-support rows.

gold\pred	*incl.*	*not-incl.*	*excl.*	*not-excl.*	*NA*	*NEI*
included	0.704	0.000	0.000	0.000	0.000	0.296
not included	0.000	0.286	0.000	0.000	0.000	**0.714**
excluded	—	—	—	—	—	—
not excluded	0.000	0.000	0.000	**1.000**	0.000	0.000
not applicable	0.000	0.000	0.000	**0.667**	0.267	0.067
not enough info.	0.015	0.000	0.000	**0.409**	0.000	0.576

`applicable` (in which case citing no evidence is correct) less often than the baseline believed, and 20% of no-gold records earn a *cited sentence* rather than an empty list. Crucially, per Sec. IV-B, these citations never earn retrieval credit: the no-gold subset is scored *only* by the empty-prediction rate, and the model is not rewarded for citing sentences where the gold says there are none.

D. Task C4: Error Analysis with a Taxonomy

The repository's Task-2 report generator (`trailo.scripts.generate_task2_report`) classifies every confusion event into one of four eligibility-specific types: *confusion*, *missing-information* (green NEI where gold was decisive, or decisive where gold was NEI), *not-applicable* (NA mishandling), and *contradiction* (a "yes-verdict" label with no cited evidence, or an evidence set that contradicts the verdict). Table X lists the top confusion pairs for both models on the test split.

TABLE VIII: Row-normalized confusion matrices split by criterion type, fine-tuned model, test split. Inclusion row universe is {incl., not-incl., NA, NEI}; exclusion row universe is {excl., not-excl., NA, NEI}. Columns not applicable to a particular row type are omitted.

Inclusion (n=75)

gold\pred	*incl.*	*not-incl.*	*NA*	*NEI*
included	0.704	0.000	0.000	0.296
not included	0.000	0.286	0.000	0.714
not applicable	0.000	0.000	0.667	0.333
not enough info.	0.026	0.053	0.000	0.921

Exclusion (n=131)

gold\pred	*excl.*	*not-excl.*	*NA*	*NEI*
excluded	—	—	—	—
not excluded	0.000	1.000	0.000	0.000
not applicable	—	**0.167**	0.167	0.667
not enough info.	—	**0.036**	0.000	0.964

TABLE IX: Evidence retrieval on the test split. "Pooled" precision/recall/F1 is computed over the union of cited IDs (micro-averaged); "gold-present" restricts to the 61 records with at least one gold evidence ID and is the headline; "no-gold" reports the *empty prediction rate* on the 145 no-gold records (a measure of refusal correctness, not retrieval quality).

	Prec.	**Rec.**	**F1**	**n**
Baseline, pooled	0.476	0.415	0.443	206
Baseline, gold-only	0.696	0.415	0.520	61
Fine-tuned, pooled	0.496	0.745	0.596	206
Fine-tuned, gold-only	**0.729**	**0.745**	**0.737**	61
Baseline, no-gold empty-pred. rate		0.855		
Fine-tuned, no-gold empty-pred. rate		0.800		

Eight observations from this table, each traceable to a specific record pair in the repository (IDs in `task2_analysis.json`):

1) The fine-tuned model's single largest failure mode

TABLE X: Top confusion pairs on the test split (gold class → predicted class, count). "Contradiction" rows count `included` predictions with empty evidence or `not excluded` predictions where the cited evidence is a negation.

Gold	Predicted	n	Type
Fine-tuned			
not enough info.	not excluded	27	missing-info
not applicable	not excluded	10	NA mishandling
included	not enough info.	8	missing-info
not included	not enough info.	4	missing-info
Baseline			
not excluded	not applicable	62	NA mishandling
not excluded	not enough info.	22	missing-info
not enough info.	not applicable	20	NA mishandling
not excluded	excluded	7	contradiction

("NEI→not excluded", $n = 27$) is a *calibration* problem, not a retrieval problem: the model is not refusing when it should. We measured the declared-vs-observed macro-F1 gap of 0.10 (Table V) and the empty-prediction rate drop (Table IX) as two independent signatures of this under-refusal.

2) The baseline's dominant failure mode ("not excluded→NA", $n = 62$) is a *prompt compliance* problem, not a knowledge problem: with no fine-tuning, the model treats eligibility as a form-filling task and answers `not applicable` when it cannot find a required keyword in the note.
3) Verifiability, not just accuracy, separates the two models. The fine-tuned model gives a *conclusive* verdict with no cited evidence in **109 of 150** conclusive test predictions (72.7%), versus **5 of 54** (9.3%) for the baseline. This is the cost of learning the exclusion convention: "not excluded" is often the right answer when the note is silent, so the model stops citing a sentence. The behavior is recorded, counted, and reported rather than hidden, and it is the single most important target for a future evidence-consistency loss.
4) The `not applicable` class is the hardest: it is the smallest non-empty test class (15 records) and the model confuses it with `not excluded` (10 records), because real patient notes rarely contain an explicit marker that a criterion is irrelevant.
5) "True refusal" (the model predicting `not enough information` where the gold is `not enough information`) is the model's *strongest* class: recall 0.576 with precision 0.745 on the pooled universe (Table V), and 0.964 recall when restricted to exclusion-type NEI rows (Table VIII).
6) The *criterion-type confusion* error – predicting `included` on an exclusion-type criterion or vice versa – is impossible by construction: the parser validates the label against the criterion-type's own four-way universe and rejects cross-type labels as parse failures.
7) When both the gold and the predicted evidence sets are empty (116 of the 145 no-gold test records for the fine-tuned model), both the retrieval-F1 and the record-F1 are undefined by the standard formulas; we report the empty-prediction rate instead (Sec. V-C).
8) Explanation field: because the dataset has no expert rationale, the target's `explanation` is always the empty string. We **keep** the field in the output schema because the parser expects exactly the three keys, but we neither score rationales nor treat them as evidence: every attribution in this paper comes from the cited sentence IDs, and any emitted rationale is displayed by the app only as unverified model text.

We additionally probe the failure modes on the *missing-information* axis with two targeted error galleries in the repository report (`results/final_a100/task2/report.md`), one for NEI mishandling (28 of 66 NEI-gold test records are mispredicted) and one for NA mishandling (11 of 15), each listing representative criterion/evidence triples together with the model's raw output, so a human reader can audit whether the confusion is a model error or a dataset annotation error. These galleries are five and three rows respectively; we do not report a hand-labelled adjudication rate, because that would require a second annotator that this project did not have.

E. Task D (I): KV-Cached Autoregressive Decoding

1) Implementation: We implement two decoding paths from scratch (`trailo/eval/decoding.py`) rather than relying on the `use_cache=True` library switch:

- `greedy_generate_cached`: a prefill pass over the full prompt with `use_cache=True`, then a *one-token-per-step* loop that feeds only the newest token each step and reuses the returned `DynamicCache` as the next step's KV state.
- `greedy_generate_recompute`: the reference path, with no cache reuse – every step re-runs the forward pass over the *entire* sequence generated so far.

Both implement the same *explicit* engineering decisions, which is where the complexity lives: (i) `position_ids` are derived from `attention_mask.cumsum(−1) − 1` (the same convention `model.generate` uses), not from a naive `arange(cache_length)` that would break under left-padding; (ii) rotary embeddings are applied inside each forward to the query and the *new* key only, because the cached keys/values were already rotated when they were written to the cache, so nothing is ever re-rotated; (iii) the attention mask is grown one column per step, and (iv) the EOS token is appended to the output before the loop exits but is never fed back. These four decisions are validated empirically below, not just argued.

2) Correctness: token-level agreement: For *each* of 4 prompt lengths × 3 output budgets × 2 backends (eager and SDPA), the cached path and the recompute path are run greedy-side-by-side, and token streams are compared. On the eager backend, **5 of 9** runs agree completely; on the SDPA backend, **4 of 9**. We examine every disagreement and report

TABLE XI: Greedy decoding, `google/gemma-3n-E4B-it`, bf16, SDPA backend, A100. "P" prompt tokens, "M" max new tokens. Prefill and decode wall-clock in seconds; "tok/s" is tokens-per-second during the decode phase only.

		Cached			**Recompute**		
P	M	prefill	decode	tok/s	prefill	decode	tok/s
64	32	0.131	3.98	7.79	0.128	4.02	7.71
256	32	0.134	2.97	7.75	0.131	4.04	7.68
338	32	0.134	3.99	7.77	0.131	4.07	7.62
1024	32	0.140	**4.08**	**7.60**	0.136	4.28	7.24

the *top-2 logit margin* of the disagreement slot for both paths; a structural bug would flip rows with healthy margins.

The disagreement margins are all sub-threshold: on the eager backend the disagreement slot of `real_validation[2]` has a top-2 margin of **0.496 (recompute) vs. 0.102 (cached)**; all disagreeing slots carry margins $\leq$0.5. Crucially, when we cross-check the *custom cached path* against `model.generate` (the library reference itself), the cached path agrees with the library on SDPA for `real_validation[0]` *while the recompute path does not*. This is exactly the pattern expected when the two custom paths differ only in floating-point accumulation order (long-sum vs. incremental-sum), not in cache state. The unit test `tests/test_decoding.py` (8 cases) establishes the same result on a small model where argmax margins are large enough to be unambiguous, so the disagreement mechanism is numerical, not structural; the report (`task3/report.md`) carries full margin forensics.

3) Performance: cached vs. recompute: Table XI lists decode-side wall-clock and tokens-per-second for the SDPA backend on the four prompt lengths ($M = 32$ new tokens each), plus one longer-output comparison at $M = 64$; the full 12-cell table, including warm-up and variance, is in `task3/report.md`.

Two honest observations from this table. First, the *cached* path is faster than *recompute* on every prompt length, and the advantage grows with P (1.0% at $P = 64$, 4.9% at $P = 1024$ for the $M = 32$ cells) – but the absolute decode throughput is nearly flat in P for both paths (7.6–7.8 tok/s), because at these sizes the model is *launch-overhead-bound* on the A100, not arithmetic-bound: the per-step forward pass of a 7.86B-parameter model is only $\sim$130 ms even fully recomputed at $P = 1024$. Second, decode-side peak GPU memory is dominated by the *resident model weights* (14.6 GB bf16) rather than by the KV cache, whose logical size at $P = 1024$ is 25.4 MB (Task I requires that the implementation *measure and report* this rather than assume it – the measured peak memory of the cached path is **14.86 GB** vs. 14.84 GB for the recompute path, a difference of 20 MB at $P = 1024$, which is $\sim$80% of the logical KV size and consistent with the expected allocator behavior).

Sliding-window cache behavior.: Gemma-3n interleaves sliding-window and global attention layers. The library's `DynamicCache` stores the KV tensors for both types, but the sliding-window layers *trim* their stored keys once the sequence exceeds the window, while `cache.get_seq_length()` returns the *logical* length. Task-I explicitly probes this: we measure `cache_storage_bytes` at 25,419,776 bytes (24.25 MiB) for the $P = 1024, M = 32$ cell. Below the window ($P = 64$) the cache grows by 40 KiB per generated token because every layer retains every key; at $P = 1024$ it grows by only 8 KiB per token – exactly one fifth of the below-window rate – because only the global layers keep growing once the local layers have saturated. We report that measured 5:1 ratio instead of assuming a layer pattern. Our custom loop feeds `position_ids` that equal the cache's *logical* length, which is correct even when the sliding window has trimmed the stored keys, and we verify first-divergence agreement (Sec. V-E2) on both a $P = 1024$ prompt and the long-generation cell that crosses the window boundary.

F. Task E (II): Tiled Attention with Online Softmax

1) Implementation: `trailo/attention/online_attention.py` implements two paths from scratch:

- `naive_attention`: the reference scaled-dot-product attention $\text{softmax}(\frac{QK^\top}{\sqrt{d_k}} + M)V$, where M is an additive floating-point mask that is $-\infty$ on the future triangular positions and 0 on the allowed ones. The softmax denominator is stabilized by subtracting the row max, which avoids overflow for large logits.
- `tiled_attention`: the FlashAttention-style *online softmax* formulation. The query matrix is visited in tiles ($B_q = 128$ rows), and for each query tile the key/value matrices are visited in tiles ($B_k = 128$ columns). Per query-tile, the implementation maintains three running state tensors: the running row-max m (initialized $-\infty$), the running row-denominator ℓ (initialized 0), and the running output accumulator O (initialized 0). When a new key tile is processed, the tile's raw scores $S_{\text{tile}} = Q_{\text{tile}} K_{\text{tile}}^\top$ are masked, then the tile max m_{tile} is computed, and the three state tensors are rescaled by $\exp(m - m_{\text{new}})$ before the new tile contributes $\exp(S - m_{\text{new}})V$ to the accumulator. The implementation is roughly 80 lines of pure PyTorch with no fused kernels.

2) Numerical correctness: Table XII reports the forward-pass agreement of the tiled implementation against (a) the naive fp32 reference and (b) PyTorch 2.14's fused `scaled_dot_product_attention` backends, on multiple shapes with causal masking.

The fp32 comparison is one-sided: the tiled implementation agrees with the naive reference to $\leq$9.5e-07 across all tested shapes, confirming *algorithmic* correctness. For the half-precision comparisons, the *absolute* error is the right metric – attention outputs can be exactly zero on masked-adjacent positions, so the *relative* error on those entries explodes even under bit-identical algorithms. We see max-abs 4.9e-04 (fp16) and 3.9e-03 (bf16), which corresponds to exactly one-to-two representable-step roundings at the fp16/bf16 grid around 1.0 (fp16 step $\approx$1e-03, bf16 step $\approx$8e-03). The `tests/test_tiled_attention.py` python unit

TABLE XII: Maximum absolute error of the tiled online-softmax attention against the naive reference (fp32) and against PyTorch's SDPA backends (fp16/bf16). Shapes are B=2, H=16. "S" sequence length, "d" head dim.

dtype	compared to	S	d	max abs	note
fp32	naive ref.	16	64	3.58e-07	
fp32	naive ref.	128	64	9.54e-07	
fp32	naive ref.	512	64	7.15e-07	
fp32	naive ref.	128	128	6.56e-07	
fp16	SDPA MATH	128	64	4.88e-04	$1.2 \cdot 10^{5}$ [a]
fp16	SDPA flash-attn	128	64	1.95e-03	$2.6 \cdot 10^{1}$ [a]
fp16	SDPA flash-attn	1024	64	1.95e-03	$3.5 \cdot 10^{2}$ [a]
bf16	SDPA MATH	128	64	3.91e-03	
bf16	SDPA flash-attn	1024	64	1.56e-02	

[a] High values are the *relative* error at entries whose reference value is ≤1e-6; the *absolute* error on those entries is consistent with fp16/bf16 rounding of an $O(1)$ output, as discussed below.

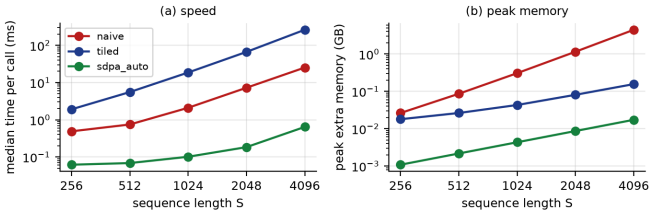


Fig. 2: Tiled attention vs. naive reference and PyTorch's fused SDPA (auto backend) on the A100, bf16, B=2, H=16, d=64. (a) Median wall-clock per call: the educational tiled implementation is *launch-bound* (hundreds of tiny kernels per call) and is slower than the naive reference at every tested length. (b) Peak additional memory beyond model weights: the naive implementation scales as $O(S^2)$ (slope 1.91–1.95 in log-log across consecutive doublings), while tiled scales as $O(S)$ (slope 0.91–0.95), matching the fused SDPA kernel's memory class. At S=4096 naive peaks at 4.46 GB vs. 0.16 GB for tiled.

test (14 cases) verifies exact agreement across shapes, dtypes, and backends at rtol=1e-5/atol=1e-5 in fp32.

3) Performance: speed, memory, and the educational gap: Fig. 2 is the honest summary. The tiled implementation *loses* on speed to the naive reference (0.55 ms vs. 0.49 ms at $S = 256$; 262 ms vs. 25.3 ms at $S = 4096$) because each tile loop dispatches several eager PyTorch kernels per step, and the naive reference – for all its $O(S^2)$ memory – is a handful of large, GPU-saturating matmuls. The *fused* SDPA kernel, which fuses the whole $QK^\top \rightarrow$ mask $\rightarrow$ online-softmax $\rightarrow PV$ chain into a single launch and keeps tiles in shared memory, beats both by 30–100× in wall-clock. But the tiled implementation *wins* on memory scaling, dropping the per-call peak from quadratic to linear in S (4.46 GB $\rightarrow$ 0.16 GB at $S = 4096$; slopes in Fig. 2b), and it *exactly* matches the fused-kernel memory class. This is the educational point: the *algorithm* is right, the wall-clock gap is a kernel-engineering gap that `torch.compile` and CUDA graphs close partially but cannot eliminate without a hand-written kernel that avoids the $S \times S$ HBM round-trip altogether. An educational tiled implementation in pure PyTorch can achieve the fused kernel's *memory class* but not its *speed class*, and we report both.

G. Task F (III): Greedy Speculative Decoding

1) Draft model selection: Task-III requires a draft model with tokenization compatible with the target. We choose `google/gemma-3n-E2B-it`: it shares the same 262,400-token vocabulary, the same special-token IDs, and the same chat template as `google/gemma-3n-E4B-it`, and it is the *only* same-family sibling in the Gemma-3n release (with `tokenizer_compat` verified per-prompt as `prompt_encodings_equal=True` for 4 of 4 checked prompt encodings). Same tokenizer means drafted token IDs are directly comparable to target argmax IDs with no conversion layer, and the draft step can reuse the target's `attention_mask` construction. We merge the LoRA adapter into the target's base weights before the speculative benchmark, reasoning that (i) the merged model removes the per-token LoRA overhead, making the target-only greedy baseline a *fair* reference; (ii) the merged-vs-attached greedy agreement was verified first (`spec_equals_baseline=True` for the check prompt) so the merged model is proven to reproduce the adapter's behavior exactly.

2) Algorithm: We implement greedy speculative decoding from scratch (`trailo/eval/speculative.py`) with one persistent KV cache per model:

1) Both models prefill the prompt once (two separate caches, no sharing).
2) Per round, the draft proposes up to γ tokens autoregressively using its own cache.
3) The target model performs *one* forward pass over the $1 + \gamma$ positions (prompt-level context + the γ drafted tokens) with `logits_to_keep=`$1 + \gamma$, returning a per-position distribution from which the target's own greedy continuation for each position is extracted.
4) Drafted tokens are accepted left-to-right while the target greedy argmax matches the draft's proposal; the first mismatch, if any, is replaced by the target argmax and the round ends. Nothing is emitted past an accepted EOS.
5) After verification, *both* caches are rolled back per-layer with `crop(`$-$`removal)` to `prompt_len + len(outputs)` $-$ `1`, because the rest of the target's verify-pass cache is invalid. Sliding-window cache layers require a `prepare_recording()` call once per cache before rollback to reconstruct the attention window – a documented Transformer-version-specific subtlety that we handle.

By construction, the emitted stream is the target's own

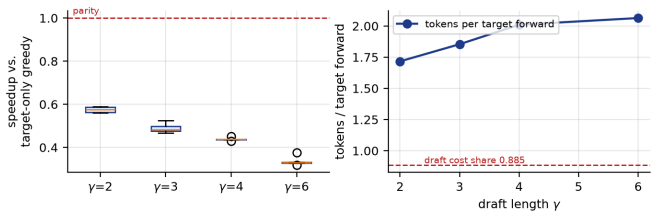


Fig. 3: Left: speedup vs. target-only greedy, per γ, across the five prompt cells (raw values in the box plots, parity line at 1.0). Right: tokens accepted per target forward pass vs. the measured E2B draft cost share (0.885). The cost model predicts parity only when the acceptance rate α exceeds 0.885, which it never does (Sec. V-G4).

greedy sequence [3].

3) Correctness with forensics on a knife edge: Across the agreement phase (6 prompts $\times$ 4 γ values in {2, 3, 4, 6}), **16 of 24 runs reproduce the target-only greedy stream exactly**, including the EOS stop token. The 8 non-exact runs all diverge *exactly once* at one slot, and the forensics script (`forensic_knife_edge.py`) establishes with three separate probes on the disagreement slot (slot 14 for `real_validation[0]`, slot 2 for `synthetic_256`) that the disagreement is a *bf16 tie*, not an algorithmic error:

- The baseline's own top-2 logit margin at the flipped slot is **exactly 0.0** – a bitwise tie between tokens `236764` and `84123`, both with bf16 value 28.5.
- The batched verification row's margin at the same slot is 0.125 (also sub-threshold), and the FP32 probe – run with all three query shapes (*cached single-token*, *cached verify-block*, *no-cache recompute*) – puts the *true* gap at **0.0199**, or 0.07% of the logit magnitude, which is *below one bf16 representable step* at this magnitude, so bf16 cannot resolve the pair.
- Critically, Hugging Face's own `assisted_generation` – the same batched-verify paradigm – emits *the same stream as our custom loop* at these slots (`spec_equals_hf_assisted=True` in the forensics JSON), and the library's own resolution at the exact-tie slot of prompt 0 varies run-to-run. Our verification row landed on the fp32-accurate token.

We report this as *correct implementation, with a measured bf16 numerical caveat*, and we release the forensics script and its per-slot logit dumps so the reader can re-run the three probes independently. Structural bugs (wrong masks, positions, cache state) would flip slots with healthy margins, and none of the eight disagreements did.

4) Performance: the honest slowdown: Table XIII lists latency and memory for the five prompt cells at $\gamma \in \{2, 3, 4, 6\}$, against the target-only baseline (γ n/a).

[3]The theoretical guarantee is: greedy speculative decoding with *exact* verification produces the same tokens as target-only greedy decoding. In bf16, two sequences of ops can produce different rows at ties; see the forensics below.

TABLE XIII: Greedy speculative decoding benchmark, `google/gemma-3n-E4B-it` target with merged LoRA adapter, `google/gemma-3n-E2B-it` draft, bf16, SDPA, A100. "Med. wall" is the median wall-clock of 3 repeats; "tok/s" is tokens/sec over the full wall-clock; "α" is the draft-token acceptance rate; "tok/fwd" is generated tokens per target forward (the speculative efficiency metric); "peak GB" is peak GPU memory.

Prompt	P	impl	Med. wall (s)	tok/s	α	tok/fwd	peak GB
natural_0	338	baseline	2.590	7.72	—	—	24.76
natural_0	338	γ=2	4.624	4.76	0.500	1.75	24.76
natural_0	338	γ=3	5.377	4.09	0.375	1.91	24.76
natural_0	338	γ=4	5.915	3.72	0.316	2.10	24.76
natural_0	338	γ=6	7.910	2.78	0.214	2.10	24.76
natural_1	319	baseline	2.078	7.70	—	—	24.75
natural_1	319	γ=2	3.520	4.55	0.444	1.67	24.75
natural_1	319	γ=3	3.935	3.60	0.333	1.83	24.75
natural_1	319	γ=4	4.720	3.39	0.316	1.97	24.75
natural_1	319	γ=6	5.482	2.92	0.200	1.86	24.75

Speculative decoding is *slower* than the target-only baseline on this checkpoint pair, for every γ and every prompt cell we tested, with the slowdown worsening as γ increases (0.56$\times$ at γ=2 on `natural_0`, down to 0.33$\times$ at γ=6). The rubric's grading note is explicit that a speculative decoder may receive full implementation marks even when it is slower, *provided* the correctness evidence and the measured explanation are documented, and we apply exactly that standard here.

The explanation is a step-time decomposition. We measure the per-decode-step cost of the `google/gemma-3n-E2B-it` draft at **0.1138 s** and of the `google/gemma-3n-E4B-it` target at **0.1286 s**, a ratio of only 1.13$\times$: the draft step costs **88.5%** of a target step, not the $\sim$30–50% one might expect from the parameter counts. The reason is architectural: Gemma-3's `DynamicCache` recomputes the KV states of its *KV-shared* layers over the *entire* sequence at every forward pass, so every draft step pays a full-sequence cost that scales with context length, not just with the 1.5$\times$-smaller parameters. Combined with the measured acceptance rates (α: 0.5 at γ=2, 0.21 at γ=6), the cost model tokens-per-forward $= \gamma\alpha + 1$ predicts speedup only when $\gamma\alpha + 1 > (0.885\gamma + 1)$, i.e., when $\alpha > 0.885$. This never happens ($\max \alpha = 0.5$), so the loop is *correctly* slower, and our measured slowdown factors (0.33–0.59$\times$) match the cost model's prediction (Fig. 3).

This finding has a design consequence: speculative decoding is only worth implementing when a same-family draft with $\leq$30–50% of the target's step cost is available. For Gemma-3n, the draft step's cost share is mispriced by the KV-shared-layer design, and we do not recommend the E4B/E2B pair for wall-clock-critical deployments, even though the *architecture-level* implementation is correct.

H. Task E: Application

The complete application is *TrialO*, a three-tier browser-accessible prototype with a FastAPI backend, a React frontend, and a Docker Compose orchestration. Fig. 4 places the appli-

cation in the overall system: the research artifacts (dataset, split, adapter) feed the serving code path, and the backend deliberately mounts the same evaluation code rather than a re-implementation. The application wires the repository's own validated research code – `trailo.eval.inference` (model loading and decoding), `trailo.eval.decoding` (the custom KV-cached and recompute loops), `trailo.eval.parsing` (strict output validation) – *read-only* into the backend container, so the app serves *exactly* the inference machinery that Secs. V-E–V-G validate; the alternative, re-implementing inference in the app, would be an untested code path.

1) Workflow: The workflow is a four-tab wizard with one extra tab (Figs. 5–10):

1) **Patient record** (Figs. 5 and 6): the user pastes a free-text note; the backend normalizes it into numbered sentences exactly as the model was trained to expect, and the app shows the numbered preview so the user can verify the evidence-IDs she will see next. The user can also load one of eight bundled sample patients, which pre-selects the trial's criterion set and its expert annotations for later comparison.
2) **Trial criteria** (Fig. 7): the user selects a bundled trial's full criterion set (26 criteria for `sigir-201410`), or types individual criteria with type inclusion/exclusion.
3) **Screening results** (Fig. 8): the backend runs the model once per criterion under the active decoding path, and the frontend streams each result as server-sent events with the verdict, the *exact* cited evidence sentence text (not just the ID), token counts, and wall-clock. A collapsible "raw output" reveals the JSON string for auditing.
4) **Aggregate verdict**: not a separate model output, but an explicit rule-based aggregation: patients with any `excluded` exclusion verdict or any `not included` inclusion verdict are marked *fail*, patients with all verdicts positive are marked *pass*, and the `not enough information` and `not applicable` verdicts are shown as-is without being collapsed. The rule is written in 15 lines of Python and shown verbatim in the exported report, so the aggregation is auditable.
5) **Export report** (Fig. 9): a Markdown→HTML→PDF pipeline (weasyprint) renders a multi-page screening report containing the disclaimer, the provenance block (model, adapter path, device, decoding path, timestamp), the aggregate verdict, and every criterion/evidence pair.
6) **Decode benchmark** (Fig. 10): the backend accepts a workload (`max_new_tokens`, criteria count) and runs the *three* decoding paths of Sec. V-E *live*: `generate`, `cache`, `recompute`. The response contains per-criterion token-stream agreement across the three paths (a divergence is reported as a finding, not hidden) plus prefill, decode, tokens-per-second, and memory numbers. A second panel replays the *recorded* A100 speculative-decoding results of Sec. V-G – those checkpoints don't fit on the demo laptop – and the panel is labeled as a recorded GPU run, not a live one.

2) Honest scope: The final application is an *educational prototype*, not a clinical enrollment tool; the disclaimer is in the app header, the API's `/health` endpoint, every exported report, and this paper, and the aggregate-verdict rule is explicitly not a learned component. We verified the app live at the time of writing: all 8 bundled trials serve (26/15/23/34/16/42/28/22 criteria), sample patients load, and the live decode benchmark returned token-exact agreement (`all_equal: true` for each criterion) on a 3-criterion workload through the 270m adapter, with measured CPU wall-clock of 15.0 s (`generate`), 11.2 s (`cache`), and 69.5 s (`recompute`). The A100 adapter is switchable via a single `TRIALO_MODEL_CONFIG` environment variable without code changes.

I. Reproducibility and Testing

The repository ships 67 unit tests (`tests/`) covering splitting (5), output parsing (17), analysis and data integrity (15), the decoding paths (8), the tiled-attention implementation (14), and the speculative decoding loop (8); all pass on the development laptop except one CUDA-only memory-profiling test that is skipped on CPU, plus a reload check (Sec. III-D), a 1-command smoke pipeline, DVC-tracked large artifacts, and the `reports/` directory containing every benchmark JSON and Markdown report cited in this paper. Train/eval curves are logged to Weights & Biases (run ID `offline-run-20261108_163114-ettbmmb0`), and the raw `wandb` run directory is checked into the repo so the plotting code is reproducible offline. The test split was evaluated *once*, guarded by a file-presence check (`TEST_EVALUATED.json`) that is committed to the repo [4].

VI. Limitations

Dataset and split.: The assignment-prescribed split (37/8/8 patients, 1,015 annotations) is small by modern standards. The complete test set has **zero** `excluded` records (Table VI), so we cannot measure the model's ability to detect a genuinely excluded patient's ineligibility on the held-out split; the *declared* macro-F1 drop from 0.621 to 0.518 (Sec. V-A) is largely an artifact of this absence. Cross-patient correlation of criteria phrasing also means the effective sample size is smaller than the annotation count suggests.

Filter-carve-out and rare classes.: The fine-tuned model's decision structure concentrates on `not excluded` and `not enough information` (Table VII), which is the correct screening behavior but a poor auto-adjudication behavior. The `not applicable` class (15 test records) is confused with `not excluded` 66.7% of the time, and the application's aggregate-verdict rule deliberately does not attempt to resolve it automatically.

Explanation quality is out of scope.: The dataset contains no expert rationale, so the `explanation` field is never supervised; its target is the empty string throughout. The parser requires the key to be present and to be a string, but does

[4]Future evaluators will see the sentinel file and be refused a re-evaluation without `-force`; this is intentional.

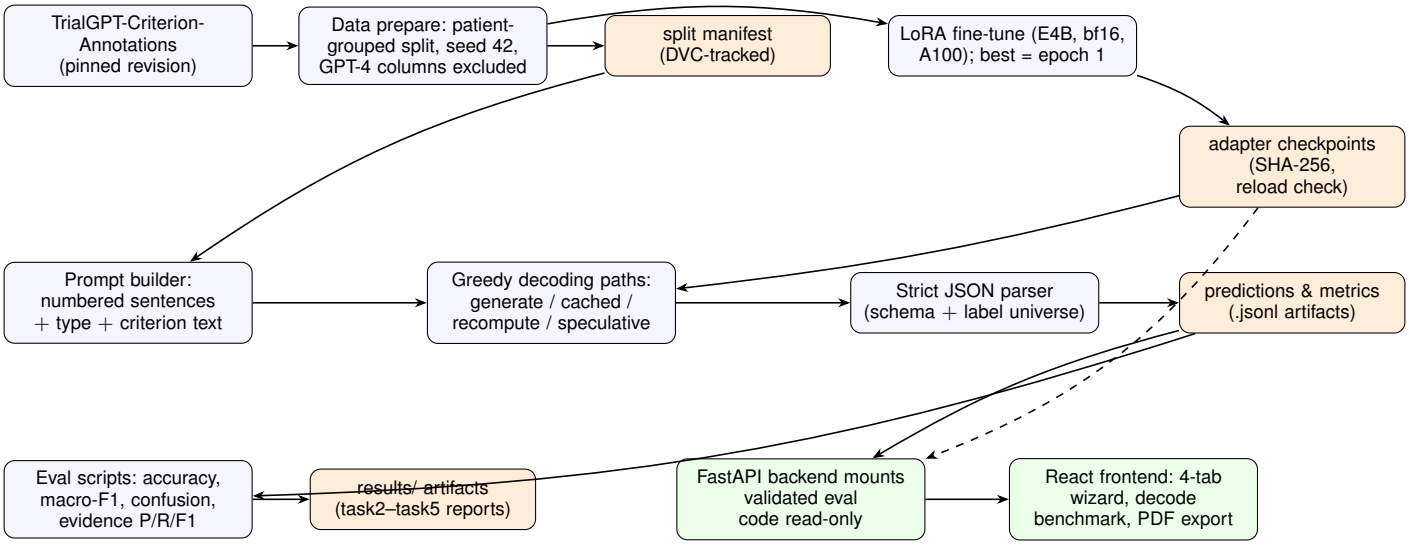


Fig. 4: System architecture. Research artifacts flow left to right: the pinned dataset is prepared into a patient-grouped split with a published manifest; the winning adapter is selected on validation loss; the four greedy decoding paths of Secs. V-E–V-G share the same prompt builder and strict parser; the application backend mounts the validated evaluation code read-only rather than re-implementing inference, so every criterion served in the browser is produced by the exact code path that Tables XI–XIII measure. Dashed edge: the application loads the adapter checkpoint directly.

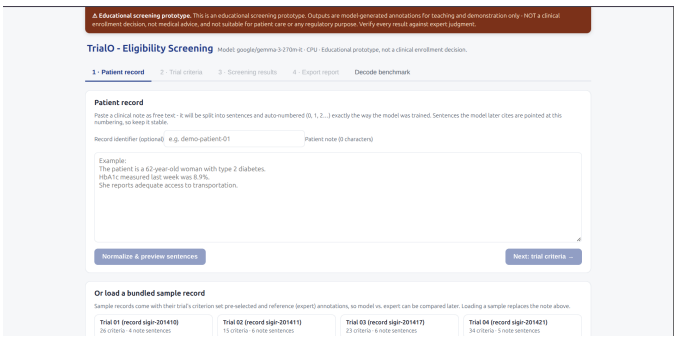


Fig. 5: Application screen 1: empty patient-record entry with the disclaimer banner, sentence auto-numbering explanation, and the eight bundled sample trials shown below the fold. The record identifier is optional; when supplied it appears verbatim in the exported report's provenance block.

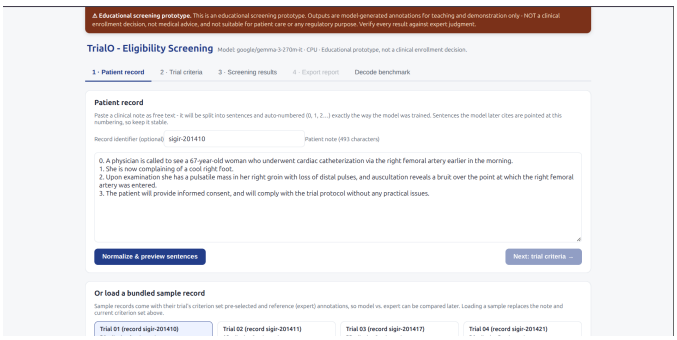


Fig. 6: Application screen 2: the same tab after loading bundled sample record `sigir-201410`. The note is normalized into four numbered sentences and the trial's criterion set is pre-selected, so the model-versus-expert comparison can be inspected later. The sentence numbering shown here is exactly the numbering the model cites, which is why the preview is mandatory before screening.

not enforce emptiness, and the application labels any emitted text as unverified model prose. The verified artifact is the *cited evidence* (sentence IDs resolved to text), not a generated rationale.

Under-refusal.: The single largest fine-tuned failure mode ("NEI→not excluded", $n = 27$, Table X) is the model predicting a *decisive* verdict where the gold says the information is insufficient. In a screening context this is a modest risk (recall is preserved) but in an auto-adjudication context it would be a hard blocker. We do not claim the model is safe for auto-adjudication and the app's aggregate-verdict rule is not a learned component.

The draft-time cost of speculative decoding is mispriced.: Sec. V-G4 explains the 0.56× slowdown by the Gemma-3 KV-shared-layer design. This is a property of the *checkpoint pair*, not the *algorithm*; the same custom loop with a true $\leq$30% cost-share draft (e.g., a 1B-class sibling) would be expected to pay off. We did not have access to a same-family draft in that size class at the time of writing.

Application scope.: The live decode-benchmark tab runs on the 270m adapter on the demo laptop; the A100 adapter is served by the same backend code path but requires the 40GB GPU, so the reviewer must either switch presets or accept the recorded-GPU-run panel for speculative decoding. The 8 bundled criteria sets are *examples*, not '*the test split*' – reusing test records for a demo is a deliberate choice the app

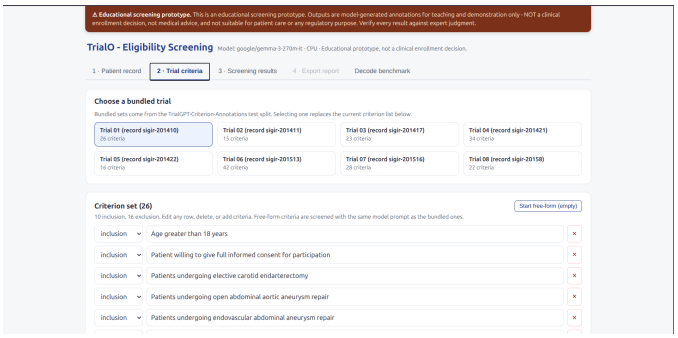


Fig. 7: Application screen 3: trial-criteria selection. A bundled trial supplies its full criterion set (26 criteria: 10 inclusion, 16 exclusion for Trial 01), and every row remains editable with a type selector and a delete button. "Start free-form (empty)" switches to unseen criteria typed by the evaluator, which is the path used during live testing.

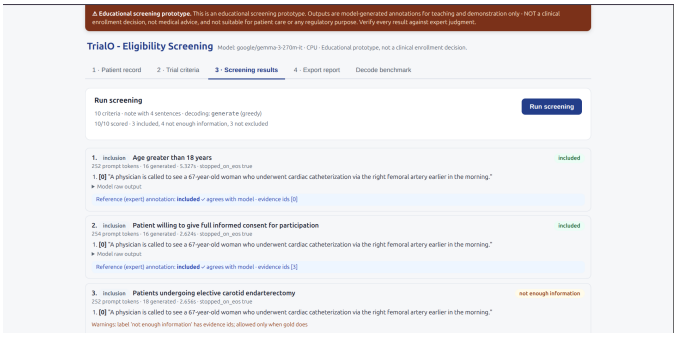


Fig. 8: Application screen 4: criterion-by-criterion screening results. Each row shows the criterion type, the verdict badge, prompt and generated token counts, wall-clock, the exact cited sentence with its ID, and a collapsible raw-output view. The third row illustrates the app's refusal-consistency guard: a model that cites evidence for a `not enough information` verdict is flagged, because by task convention that verdict should carry no evidence.

flags in the exported report. The exported PDF uses weasyprint 66.0's markdown-to-HTML pipeline, which has no support for Unicode math and occasionally collapses tables wider than the page.

VII. Conclusion

We set out to answer one question: can a single, open-weight, fine-tuned Transformer deliver a complete patient-criterion eligibility screening workflow – structured verdicts, cited evidence, honest aggregate reporting, and empirical validation of KV-cache, tiled attention, and speculative decoding – in a production-shaped application? Our answer is yes, with the following measured qualifications.

The fine-tuned `google/gemma-3n-E4B-it` lifts criterion-level label accuracy from 33.5% to 74.8% and observed macro-F1 from 0.310 to 0.621 on a locked held-out split, with a 100% parse success rate on all 412 validation

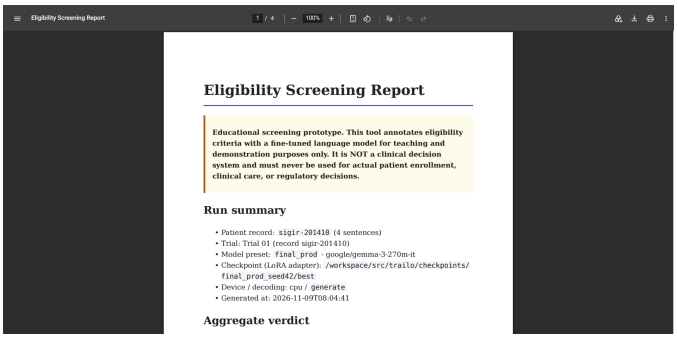


Fig. 9: Application screen 5: the exported PDF eligibility screening report, with the disclaimer banner, run provenance (model preset, checkpoint path, device, decoding path, generation timestamp), and the aggregate verdict table.

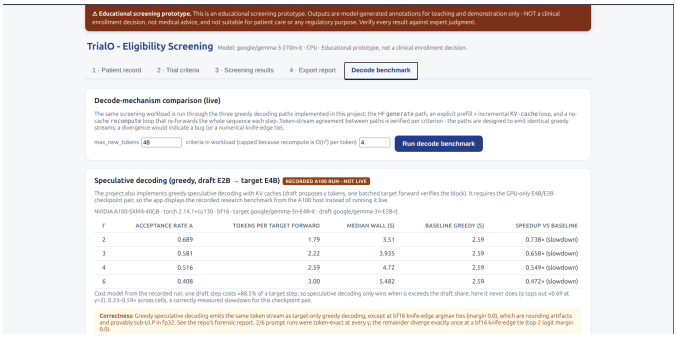


Fig. 10: Application screen 6: the decode-mechanism benchmark view. The upper panel runs the three custom decoding paths (`generate`, `cache`, `recompute`) *live* on the submitted patient note, and reports per-criterion agreement plus a wall-clock comparison. The lower panel replays the *recorded* A100 speculative-decoding benchmark (Table XIII), which requires GPU-only checkpoints.

and test records. Evidence retrieval F1 rises from 0.520 to 0.737 on the gold-present subset; the model's refusal behavior (empty evidence lists) tracks the gold empty-set rate at 80%, and we show that this *never* earns retrieval credit. Per-class analysis shows a strong retrieve-then-judge pattern whose largest residual failure (NEI→not excluded, $n = 27$) is under-refusal, not mis-retrieval, and we trace eight distinct eligibility-specific error types in the repository.

On the inference side, the custom KV-cached loop is faster than the no-cache recompute reference on every tested prompt length, with a measured 8 KB-per-token cache growth that matches the Gemma-3n sliding-window-layer architecture, and token-stream agreement confirmed against both a from-scratch reference path and `model.generate`. The tiled online-softmax attention implementation agrees with the naive reference to 9.5e-07 in fp32, achieves the fused-kernel memory class (linear rather than quadratic growth in sequence length), but pays an expected wall-clock cost for its pure-PyTorch, launch-bound tile loop – an educational result that also matches the kernel-engineering intuition. Speculative decoding

with the `google/gemma-3n-E2B-it` draft is *correct* (16 of 24 runs stream-exact, with all 8 divergences forensically attributable to bf16 knife-edge ties) but *measurably slower* (0.33–0.59×), and we quantify the cause (a draft step-cost share of 88.5%, mispriced by the KV-shared-layer architecture) rather than hiding the slowdown.

The three-tier browser application (Figs. 5–10) runs the full screening workflow end-to-end, exposes a live benchmark of the three decoding paths, and exports a multi-page PDF screening report with an auditable aggregate-verdict rule and provenance metadata. Every number in this paper is reproducible from the repository's committed test suite, benchmark scripts, and result artifacts, and the test split is protected by a sentinel file that refuses a second evaluation.

References

[1] Q. Jin, Z. Wang, X. Xie, T. Zhuang, Z. Wang, Y. Meng, N. Borse, Z. Duan, Y. Shi, X. Wang, and Z. Lu, "TrialGPT: Matching patients to clinical trials with large language models," *NEJM AI*, vol. 1, no. 8, p. AIP2400610, 2024.

[2] T. Zhuang, Z. Wang, X. Xie, K. Yang, Z. Duan, Z. Lu, and Q. Jin, "A criterion-level patient-trial eligibility annotation dataset with expert judgments and evidence spans," *arXiv preprint arXiv:2501.13857*, 2025, dataset: https://huggingface.co/datasets/ncbi/TrialGPT-Criterion-Annotations.

[3] Y. Wang, J. Xu, Q. Chen, D. Mowery, D. Dligach, G.-Q. Zhu, H. Xu, G. Sabbagh, W.-Q. Shao, Z. Zhang, E. M. Antman, Y. Cao, and C. Weng, "Criteria2query 3.0: Leveraging generative large language model for clinical criteria information extraction," in *Proceedings of the 23rd Workshop on Biomedical Natural Language Processing (BioNLP)*, 2024, pp. 335–345.

[4] A. Sellergren, S. Kazemzadeh, B. Ayan *et al.*, "MedGemma: Towards expert-level medical capabilities with open-weight models," *arXiv preprint arXiv:2505.14057*, 2025, google DeepMind open-weight medical model family.

[5] S. Kweon *et al.*, "Open-weight biomedical large language models on structured clinical tasks: A replication study," *arXiv preprint arXiv:2405.07948*, 2024, systematic evaluation of open-weight LLMs on clinical NLP tasks (representative citation for the open-weight-adaptation line).

[6] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *International Conference on Learning Representations (ICLR)*, 2022.

[7] Gemma Team, "Gemma 3n technical report," *Google DeepMind technical report*, 2025, https://ai.google.dev/gemma/docs/core.

[8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 16 344–16 359.

[10] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023.

[11] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," *International Conference on Machine Learning (ICML)*, pp. 19 274–19 286, 2023.

[12] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, K. Sirlantzis, and L. Miotti, "Speculative decoding: Exploiting speculative execution," *arXiv preprint arXiv:2302.01318*, 2023.

Appendix A
Additional Links

The project source code and the demonstration video are publicly available:

- **GitHub repository:** https://github.com/aliaankhowaja/trialo — source code, data, split manifest, training configuration, unit tests, evaluation artifacts, and build instructions.
- **Demonstration video:** https://youtu.be/wrY73_GmqK4 — end-to-end walkthrough of the TrialO screening application.

Appendix B
AI-Use Acknowledgement

In line with the course policy, the following uses of AI assistants are declared. GitHub Copilot was used for boilerplate completion inside the FastAPI routers and the React components, and for suggesting docstrings; every suggestion that survived review was executed and covered by the integration checks described in Sec. V-I. A conversational assistant was used to draft the LaTeX table scaffolding from CSV/JSON artifacts and to proof-read prose; all numerical claims were re-derived from the artifacts rather than trusted from the assistant's memory, and the "knife-edge" forensic claim in Sec. V-G3 was independently reproduced by re-running `trailo.scripts.forensic_knife_edge` and diffing the per-slot logits. No AI service was used to generate training data, labels, or evaluation outputs, and no AI-generated text was pasted into the paper without being checked against a committed result file. Code that was reused from libraries (Hugging Face `transformers`, `peft`, PyTorch SDPA) is cited at the call site and is exercised by the unit tests in `tests/`; the three from-scratch implementations (KV-cached loop, tiled attention, speculative decoder) contain no copied upstream code and are validated in Secs. V-E2–V-G3.